

Semantic Analysis

se-man-tic: of or relating to meaning in language

Webster's Dictionary

The *semantic analysis* phase of a compiler connects variable definitions to their uses, checks that each expression has a correct type, and translates the abstract syntax into a simpler representation suitable for generating machine code.

5.1

SYMBOL TABLES

This phase is characterized by the maintenance of *symbol tables* (also called *environments*) mapping identifiers to their types and locations. As the declarations of types, variables, and functions are processed, these identifiers are bound to “meanings” in the symbol tables. When *uses* (nondefining occurrences) of identifiers are found, they are looked up in the symbol tables.

Each local variable in a program has a *scope* in which it is visible. For example, in a MiniJava method m , all formal parameters and local variables declared in m are visible only until the end of m . As the semantic analysis reaches the end of each scope, the identifier bindings local to that scope are discarded.

An environment is a set of *bindings* denoted by the $\mapsto$ arrow. For example, we could say that the environment σ_0 contains the bindings $\{g \mapsto \text{string}, a \mapsto \text{int}\}$, meaning that the identifier a is an integer variable and g is a string variable.

Consider a simple example in the Java language:

```

1 class C {
2   int a; int b; int c;
3   public void m(){
4     System.out.println(a+c);
5     int j = a+b;
6     String a = "hello";
7     System.out.println(a);
8     System.out.println(j);
9     System.out.println(b);
10  }
11 }

```

Suppose we compile this class in the environment σ_0 . The field declarations on line 2 give us the table σ_1 equal to $\sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$, that is, σ_0 extended with new bindings for a , b , and c . The identifiers in line 4 can be looked up in σ_1 . At line 5, the table $\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$ is created; and at line 6, $\sigma_3 = \sigma_2 + \{a \mapsto \text{String}\}$ is created.

How does the $+$ operator for tables work when the two environments being “added” contain different bindings for the same symbol? When σ_2 and $\{a \mapsto \text{String}\}$ map a to int and String , respectively? To make the scoping rules work the way we expect them to in real programming languages, we want $\{a \mapsto \text{String}\}$ to take precedence. So we say that $X + Y$ for tables is not the same as $Y + X$; bindings in the right-hand table override those in the left.

The identifiers in lines 7, 8, and 9 can be looked up in σ_3 . Finally, at line 10, we discard σ_3 and go back to σ_1 . And at line 11 we discard σ_1 and go back to σ_0 .

How should this be implemented? There are really two choices. In a *functional* style, we make sure to keep σ_1 in pristine condition while we create σ_2 and σ_3 . Then when we need σ_1 again, it’s rested and ready.

In an *imperative* style, we modify σ_1 until it becomes σ_2 . This *destructive update* “destroys” σ_1 ; while σ_2 exists, we cannot look things up in σ_1 . But when we are done with σ_2 , we can *undo* the modification to get σ_1 back again. Thus, there is a single global environment σ which becomes $\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_1, \sigma_0$ at different times and an “undo stack” with enough information to remove the destructive updates. When a symbol is added to the environment, it is also added to the undo stack; at the end of scope (e.g., at line 10), symbols popped from the undo stack have their latest binding removed from σ (and their previous binding restored).

Either the functional or imperative style of environment management can be used regardless of whether the language being compiled or the implemen-

5.1. SYMBOL TABLES

```
structure M = struct
  structure E = struct
    val a = 5;
  end
  structure N = struct
    val b = 10
    val a = E.a + b
  end
  structure D = struct
    val d = E.a + N.a
  end
end
```

(a) An example in ML

```
package M;
class E {
  static int a = 5;
}
class N {
  static int b = 10;
  static int a = E.a + b;
}
class D {
  static int d = E.a + N.a;
}
```

(b) An example in Java

FIGURE 5.1. Several active environments at once.

tation language of the compiler is a “functional” or “imperative” or “object-oriented” language.

MULTIPLE SYMBOL TABLES

In some languages there can be several active environments at once: Each module, or class, or record in the program has a symbol table σ of its own.

In analyzing Figure 5.1, let σ_0 be the base environment containing pre-defined functions, and let

$$\begin{aligned}\sigma_1 &= \{a \mapsto \text{int}\} \\ \sigma_2 &= \{E \mapsto \sigma_1\} \\ \sigma_3 &= \{b \mapsto \text{int}, a \mapsto \text{int}\} \\ \sigma_4 &= \{N \mapsto \sigma_3\} \\ \sigma_5 &= \{d \mapsto \text{int}\} \\ \sigma_6 &= \{D \mapsto \sigma_5\} \\ \sigma_7 &= \sigma_2 + \sigma_4 + \sigma_6\end{aligned}$$

In ML, the N is compiled using environment $\sigma_0 + \sigma_2$ to look up identifiers; D is compiled using $\sigma_0 + \sigma_2 + \sigma_4$, and the result of the analysis is $\{M \mapsto \sigma_7\}$.

In Java, forward reference is allowed (so inside N the expression $D.d$ would be legal), so E , N , and D are all compiled in the environment σ_7 ; for this program the result is still $\{M \mapsto \sigma_7\}$.

```

class Bucket {String key; Object binding; Bucket next;
    Bucket(String k, Object b, Bucket n) {key=k; binding=b; next=n;}
}

class HashT {
    final int SIZE = 256;
    Bucket table[] = new Bucket[SIZE];

    private int hash(String s) {
        int h=0;
        for(int i=0; i<s.length(); i++)
            h=h*65599+s.charAt(i);
        return h;
    }

    void insert(String s, Binding b) {
        int index=hash(s)%SIZE
        table[index]=new Bucket(s,b,table[index]);
    }

    Object lookup(String s) {
        int index=hash(s)%SIZE
        for (Binding b = table[index]; b!=null; b=b.next)
            if (s.equals(b.key)) return b.binding;
        return null;
    }

    void pop(String s) {
        int index=hash(s)%SIZE
        table[index]=table[index].next;
    }
}

```

PROGRAM 5.2. Hash table with external chaining.

EFFICIENT IMPERATIVE SYMBOL TABLES

Because a large program may contain thousands of distinct identifiers, symbol tables must permit efficient lookup.

Imperative-style environments are usually implemented using hash tables, which are very efficient. The operation $\sigma' = \sigma + \{a \mapsto \tau\}$ is implemented by inserting τ in the hash table with key a . A simple *hash table with external chaining* works well and supports deletion easily (we will need to delete $\{a \mapsto \tau\}$ to recover σ at the end of the scope of a).

Program 5.2 implements a simple hash table. The i th bucket is a linked list of all the elements whose keys hash to $i \bmod \text{SIZE}$.

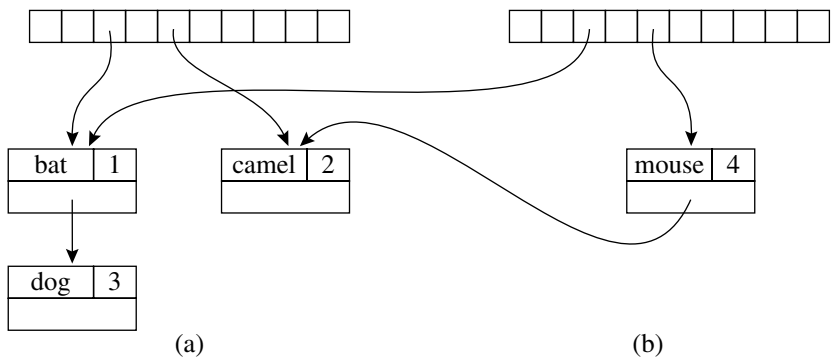


FIGURE 5.3. Hash tables.

Consider $\sigma + \{a \mapsto \tau_2\}$ when σ contains $a \mapsto \tau_1$ already. The `insert` function leaves $a \mapsto \tau_1$ in the bucket and puts $a \mapsto \tau_2$ earlier in the list. Then, when `pop(a)` is done at the end of a 's scope, σ is restored. Of course, `pop` works only if bindings are inserted and popped in a stacklike fashion.

An industrial-strength implementation would improve on this in several ways; see Exercise 5.1.

EFFICIENT FUNCTIONAL SYMBOL TABLES

In the functional style, we wish to compute $\sigma' = \sigma + \{a \mapsto \tau\}$ in such a way that we still have σ available to look up identifiers. Thus, instead of “altering” a table by adding a binding to it we create a new table by computing the “sum” of an existing table and a new binding. Similarly, when we add $7 + 8$ we don't alter the 7 by adding 8 to it; we create a new value, 15 – and the 7 is still available for other computations.

However, nondestructive update is not efficient for hash tables. Figure 5.3a shows a hash table implementing mapping m_1 . It is fast and efficient to add *mouse* to the fifth slot; just make the *mouse* record point at the (old) head of the fifth linked list, and make the fifth slot point to the *mouse* record. But then we no longer have the mapping m_1 : We have destroyed it to make m_2 . The other alternative is to copy the array, but still share all the old buckets, as shown in Figure 5.3b. But this is not efficient: The array in a hash table should be quite large, proportional in size to the number of elements, and we cannot afford to copy it for each new entry in the table.

By using binary search trees we can perform such “functional” additions to search trees efficiently. Consider, for example, the search tree in Figure 5.4,

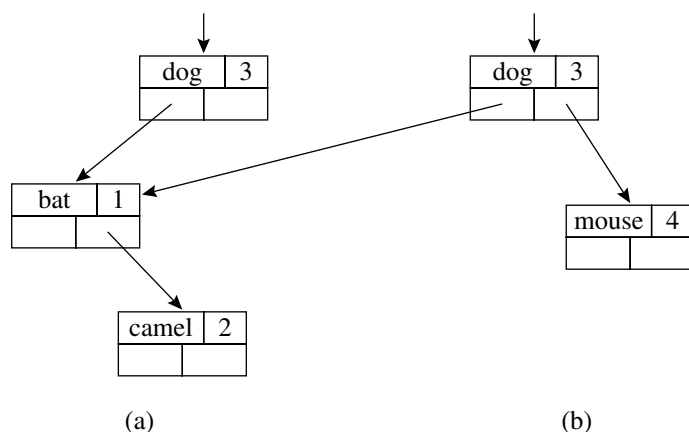


FIGURE 5.4. Binary search trees.

which represents the mapping

$$m_1 = \{bat \mapsto 1, camel \mapsto 2, dog \mapsto 3\}.$$

We can add the binding $mouse \mapsto 4$, creating the mapping m_2 without destroying the mapping m_1 , as shown in Figure 5.4b. If we add a new node at depth d of the tree, we must create d new nodes – but we don’t need to copy the whole tree. So creating a new tree (that shares some structure with the old one) can be done as efficiently as looking up an element: in $\log(n)$ time for a balanced tree of n nodes. This is an example of a *persistent data structure*; a persistent *red-black* tree can be kept balanced to guarantee $\log(n)$ access time (see Exercise 1.1c, and also page 276).

SYMBOLS

The hash table of Program 5.2 must examine every character of the string s for the hash operation, and then again each time it compares s against a string in the i th bucket. To avoid unnecessary string comparisons, we can convert each string to a `symbol`, so that all the different occurrences of any given string convert to the same symbol object.

The `Symbol` module implements symbols and has these important properties:

- Comparing symbols for equality is fast (just pointer or integer comparison).
- Extracting an integer hash key is fast (in case we want to make a hash table mapping symbols to something else).

```
package Symbol;

public class Symbol {
    public String toString();
    public static Symbol symbol(String s);
}

public class Table {
    public Table();
    public void put(Symbol key, Object value);
    public Object get(Symbol key);
    public void beginScope();
    public void endScope();
    public java.util Enumeration keys();
}
```

PROGRAM 5.5. The interface of package `Symbol`.

- Comparing two symbols for “greater-than” (in some arbitrary ordering) is fast (in case we want to make binary search trees).

Even if we intend to make functional-style environments mapping symbols to bindings, we can use a destructive-update hash table to map strings to symbols: We need this to make sure the second occurrence of “abc” maps to the same symbol as the first occurrence. Program 5.5 shows the interface of the `Symbol` module.

Environments are implemented in the `Symbol.Table` class as Tables mapping Symbols to bindings. We want different notions of binding for different purposes in the compiler – type bindings for types, value bindings for variables and functions – so we let the bindings be `Object`, though in any given table every binding should be a type binding, or every binding should be a value binding, and so on.

To implement the `Symbol` class (Program 5.6), we rely on the `intern()` method of the `java.lang.String` class to give us a unique object for any given character sequence; we can map from `Symbol` to `String` by having each symbol contain a string variable, but the reverse mapping must be done using a hash table (we use `java.util.Hashtable`).

To handle the “undo” requirements of destructive update, the interface function `beginScope` remembers the current state of the table, and `endScope` restores the table to where it was at the most recent `beginScope` that has not already been ended.

An imperative table is implemented using a hash table. When the binding

```
package Symbol;
public class Symbol {
    private String name;
    private Symbol(String n) {name=n; }
    private static java.util.Dictionary dict = new java.util.Hashtable();

    public String toString() {return name;}

    public static Symbol symbol(String n) {
        String u = n.intern();
        Symbol s = (Symbol)dict.get(u);
        if (s==null) {s = new Symbol(u); dict.put(u,s); }
        return s;
    }
}
```

PROGRAM 5.6. Symbol table implementation.

$x \mapsto b$ is entered (`table.put(x,b)`), x is hashed into an index i , and a `Binder` object $x \mapsto b$ is placed at the head of the linked list for the i th bucket. If the table had already contained a binding $x \mapsto b'$, that would still be in the bucket, hidden by $x \mapsto b$. This is important because it will support the implementation of *undo* (`beginScope` and `endScope`).

The key x is not a character string, but is the `Symbol` object itself.

There must also be an auxiliary stack, showing in what order the symbols were “pushed” into the symbol table. When $x \mapsto b$ is entered, then x is pushed onto this stack. A `beginScope` operation pushes a special marker onto the stack. Then, to implement `endScope`, symbols are popped off the stack down to and including the topmost marker. As each symbol is popped, the head binding in its bucket is removed.

The auxiliary stack can be integrated into the `Binder` by having a global variable `top` showing the most recent `Symbol` bound in the table. Then “pushing” is accomplished by copying `top` into the `prevtop` field of the `Binder`. Thus, the “stack” is threaded through the binders.

If we wanted to use functional-style symbol tables, the `Table` interface might look like this:

```
public class Table {
    public Table();
    public Table put(Symbol key, Object value);
    public Object get(Symbol key);
    public java.util Enumeration keys();
}
```

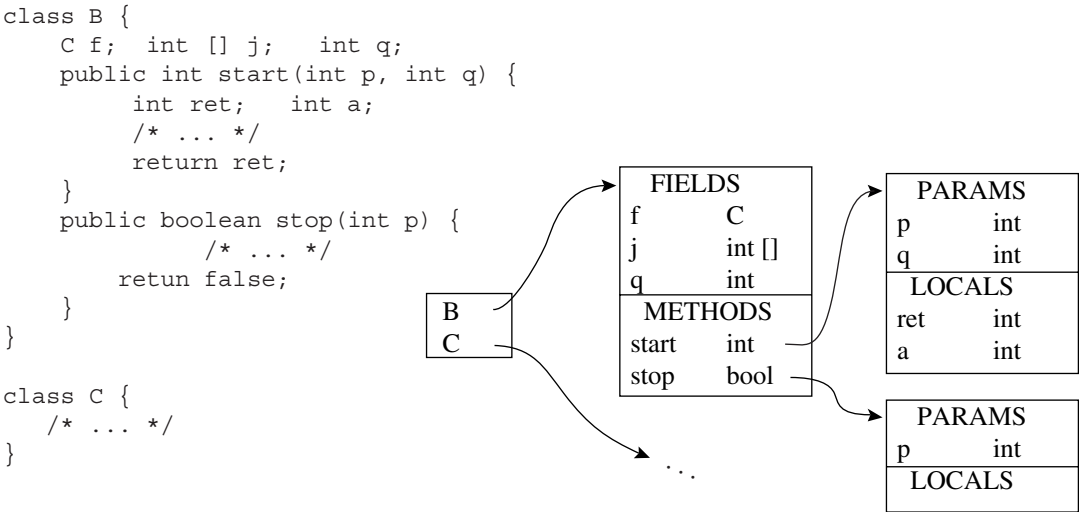



FIGURE 5.7. A MiniJava Program and its symbol table

The put function would return a new table without modifying the old one. We wouldn't need beginScope and endScope, because we could keep an old version of the table even as we use the new version.

5.2 TYPE-CHECKING MiniJava

With what should a symbol table be filled – that is, what is a binding? To enable type-checking of MiniJava programs, the symbol table should contain all declared type information:

- each variable name and formal-parameter name should be bound to its type;
- each method name should be bound to its parameters, result type, and local variables; and
- each class name should be bound to its variable and method declarations.

For example, consider Figure 5.7, which shows a program and its symbol table. The two class names B and C are each mapped to two tables for fields and methods. In turn, each method is mapped to both its result type and tables with its formal parameters and local variables.

The primitive types in MiniJava are `int` and `boolean`; all other types are either integer array, written `int []`, or class names. For simplicity, we

```

class ErrorMsg {
    boolean anyErrors;
    void complain(String msg) {
        anyErrors = true;
        System.out.println(msg);
    }
}

// Type t;
// Identifier i;
public void visit(VarDecl n) {

    Type t = n.t.accept(this);
    String id = n.i.toString();

    if (currMethod == null) {
        if (!currClass.addVar(id,t))
            error.complain(id + "is already defined in " + currClass.getId());
    } else if (!currMethod.addVar(id,t))
        error.complain(id + "is already defined in "
            + currClass.getId() + "." + currMethod.getId());
}

```

PROGRAM 5.8. A visit method for variable declarations

choose to represent each type as a string, rather than as a symbol; this allows us to test type equality by doing string comparison.

Type-checking of a MiniJava program proceeds in two phases. First, we build the symbol table, and then we type-check the statements and expressions. During the second phase, the symbol table is consulted for each identifier that is found. It is convenient to use two phases because, in Java and MiniJava, the classes are mutually recursive. If we tried to do type-checking in a single phase, then we might need to type-check a call to a method that is not yet entered into the symbol table. To avoid such situations, we use an approach with two phases.

The first phase of the type-checker can be implemented by a visitor that visits nodes in a MiniJava syntaxtree and builds a symbol table. For instance, the visit method in Program 5.8 handles variable declarations. It will add the variable name and type to a data structure for the current class which later will be added to the symbol table. Notice that the visit method checks whether a variable is declared more than once and, if so, then it prints an appropriate error message.

```
// Exp e1,e2;
public Type visit(Plus n) {
    if (! (n.e1.accept(this) instanceof IntegerType) )
        error.complain("Left side of LessThan must be of type integer");
    if (! (n.e2.accept(this) instanceof IntegerType) )
        error.complain("Right side of LessThan must be of type integer");
    return new IntegerType();
}
```

PROGRAM 5.9. A visit method for plus expressions

The second phase of the type-checker can be implemented by a visitor that type-checks all statements and expressions. The result type of each visit method is `String`, for representing MiniJava types. The idea is that when the visitor visits an expression, then it returns the type of that expression. If the expression does not type-check, then the type-check is terminated with an error message.

Let's take a simple case: an addition expression $e_1 + e_2$. In MiniJava, both operands must be integers (the type-checker must check this) and the result will be an integer (the type-checker will return this type). The visit method for addition is easy to implement; see Program 5.9.

In most languages, addition is *overloaded*: The $+$ operator stands for either integer addition or real addition. If the operands are both integers, the result is integer; if the operands are both real, the result is real. And in many languages if one operand is an integer and the other is real, the integer is implicitly converted into a real, and the result is real. Of course, the compiler will have to make this conversion explicit in the machine code it generates.

For an assignment statement, it must be checked that the left-hand side and the right-hand side have the same type. When we allow extension of classes, the requirement is less strict: It is sufficient to check that the right-hand side is a subtype of the left-hand side.

For method calls, it is necessary to look up the method identifier in the symbol table to get the parameter list and the result type. For a call $e.m(\dots)$, where e has type C , we look up the definition of m in class C . The parameter types must then be matched against the arguments in the function-call expression. The result type of the method becomes the type of the method call as a whole.

Every kind of statement and expression has its own type-checking rules, but in all the cases we have not already described, the rules can be derived by reference to the Java Language Specification.

ERROR HANDLING

When the type-checker detects a type error or an undeclared identifier, it should print an error message and continue – because the programmer would like to be told of all the errors in the program. To recover after an error, it's often necessary to build data structures as if a valid expression had been encountered. For example, type-checking

```
{int i = new C();  
  int j = i + i;  
  ...  
}
```

even though the expression `new C()` doesn't match the type required to initialize an integer, it is still useful to enter `i` in the symbol table as an integer so that the rest of the program can be type-checked.

If the type-checking phase detects errors, then the compiler should not produce a compiled program as output. This means that the later phases of the compiler – translation, register allocation, etc. – will not be executed. It will be easier to implement the later phases of the compiler if they are not required to handle invalid inputs. Thus, if at all possible, all errors in the input program should be detected in the front end of the compiler (parsing and type-checking).

PROGRAM

TYPE-CHECKING

Design a set of visitors which type-checks a MiniJava program and produces any appropriate error messages about mismatching types or undeclared identifiers.

EXERCISES

- 5.1** Improve the hash table implementation of Program 5.2: Double the size of the array when the average bucket length grows larger than 2 (so `table` is now a pointer to a dynamically allocated array). To double an array, allocate a bigger one and rehash the contents of the old array; then discard the old array.

*****5.2** In many applications, we want a $+$ operator for environments that does more than add one new binding; instead of $\sigma' = \sigma + \{a \mapsto \tau\}$, we want $\sigma' = \sigma_1 + \sigma_2$, where σ_1 and σ_2 are arbitrary environments (perhaps overlapping, in which case bindings in σ_2 take precedence).

We want an efficient algorithm and data structure for environment “adding.” Balanced trees can implement $\sigma + \{a \mapsto \tau\}$ efficiently (in $\log(N)$ time, where N is the size of σ), but take $O(N)$ to compute $\sigma_1 + \sigma_2$ if σ_1 and σ_2 are both about size N .

To abstract the problem, solve the general nondisjoint integer-set union problem. The input is a set of commands of the form,

$s_1 = \{4\}$ (*define singleton set*)
 $s_2 = \{7\}$
 $s_3 = s_1 \cup s_2$ (*nondestructive union*)
 $6 \stackrel{?}{\in} s_3$ (*membership test*)
 $s_4 = s_1 \cup s_3$
 $s_5 = \{9\}$
 $s_6 = s_4 \cup s_5$
 $7 \stackrel{?}{\in} s_2$

An efficient algorithm is one that can process an input of N commands, answering all membership queries, in less than $o(N^2)$ time.

- *a. Implement an algorithm that is efficient when a typical set union $a \leftarrow b \cup c$ has b much smaller than c [Brown and Tarjan 1979].
- ***b.** Design an algorithm that is efficient even in the worst case, or prove that this can't be done (see Lipton et al. [1997] for a lower bound in a restricted model).